Performance Analysis and Optimization of CNN Training from Scratch Using CUDA

December 31, 2025

Project: GPU Programming Project (CNN Training Pipeline in CUDA)
Team Members: Shiva Zare, Moein Yeganeh

1 Overview

We will implement and train a small Convolutional Neural Network (CNN) entirely in CUDA, with a primary emphasis on GPU performance rather than model complexity. The CNN must train correctly (loss decreases and accuracy increases), but our main objective is to analyze how CNN training workloads map onto GPU hardware. We will measure kernel-level performance, identify bottlenecks using profiling tools, and implement at least one optimization that provides a measured speedup with a clear explanation for why it improves performance.

2 Dataset

We will use the **MNIST** dataset (28×28 grayscale digits, 10 classes). MNIST is chosen because it allows fast iteration and quick correctness validation while still providing meaningful convolution workloads that can be profiled and optimized. CPU-side work will be limited to dataset loading, normalization (e.g., dividing by 255), mini-batch creation, and transferring batches to the GPU.

3 CNN Architecture

We will implement a compact LeNet-style CNN that is small enough to implement fully from scratch but still exposes realistic performance behavior in convolution and backpropagation.This network is designed so that convolution layers and their gradients dominate runtime, making performance measurement and optimization meaningful.

Architecture

Stage	Layer	Output Shape (Batch = B)
Input	–	$B \times 1 \times 28 \times 28$
Block 1	Conv($1 \to 8$, 3×3, pad=1) + Bias + ReLU	$B \times 8 \times 28 \times 28$
	MaxPool(2×2, stride=2)	$B \times 8 \times 14 \times 14$
Block 2	Conv($8 \to 16$, 3×3, pad=1) + Bias + ReLU	$B \times 16 \times 14 \times 14$
	MaxPool(2×2, stride=2)	$B \times 16 \times 7 \times 7$
Head	FC($16 \cdot 7 \cdot 7 \to 128$) + ReLU	$B \times 128$
	FC($128 \to 10$)	$B \times 10$ (logits)
Output	Softmax + Cross-Entropy Loss	loss + gradient

4 What "From Scratch in CUDA" Includes

All core training operations will be implemented as CUDA kernels (no deep learning frameworks for computation). We will implement:

- **Forward pass kernels:** convolution, ReLU, maxpool, fully connected, softmax.
- **Loss kernel:** cross-entropy (numerically stable).
- **Backprop kernels:**
  - softmax + cross-entropy gradient (preferably fused),
  - fully connected backward (dX, dW, db),
  - maxpool backward (using saved argmax indices),
  - ReLU backward,
  - convolution backward (dX, dW, db).
- **Weight update kernel:** SGD updates on GPU ($W \leftarrow W - \eta dW$, $b \leftarrow b - \eta db$).
- **Initialization:**CPU initialization and copy to device.

CPU is used only for loading/preprocessing and driving the training loop.

5 GPU Performance Assumptions (Expected Bottlenecks)

The dominant cost is expected to be (1) convolution forward and (2) convolution backpropagation (dX and dW). Naive convolution is expected to perform many redundant global memory reads because neighboring output pixels reuse overlapping input regions. Convolution dW can be particularly expensive if implemented with high-contention atomics or poorly structured reductions. These characteristics make the project suitable for studying memory bandwidth utilization, reuse, and kernel launch configuration tradeoffs.

6 Planned Optimizations (Measured Comparisons)

We will implement baseline and optimized variants and compare them using kernel timings and profiling evidence:

1. **Convolution v1 (baseline):** naive direct convolution using global memory loads.
2. **Convolution v2 (optimized):** shared-memory tiled convolution to reuse input tiles and reduce global memory traffic. If tiling provides limited improvement, we will explore memory coalescing optimization as alternatives, guided by profiling metrics.
3. **Kernel fusion (optimization):** fuse **Conv + Bias + ReLU** in the forward pass to reduce intermediate global reads/writes and reduce launch overhead.
4. dW **strategy improvement (if time allows):** reduce atomic contention using block-level partial accumulation (partial sums) followed by a merge/reduction step.

We will tune batch size and tile/block dimensions to evaluate occupancy vs. memory behavior.

7 Evaluation Plan

7.1 Correctness

- We will train for multiple epochs and show decreasing loss and increasing test accuracy.
- We will report accuracy per epoch.
- We will use small debug tests (tiny inputs/batches) to validate shapes and gradient flow.

7.2 Performance

- We will use `cudaEvent` timing to measure major kernels: conv forward, conv dX, conv dW, FC forward/backward, pool forward/backward, softmax/loss, and update kernels.
- We will report time per epoch, images/sec throughput, and a kernel time breakdown (percentage of total time).
- We will demonstrate at least one measured speedup (e.g., naive conv $\rightarrow$ tiled conv) and explain the observed change using profiling metrics.

8 Tools / Profiling

- **Nsight Systems:** timeline-level view (kernel launches, overlap, CPU/GPU interaction).
- **Nsight Compute:** kernel metrics (occupancy, memory throughput, warp efficiency, stall reasons).
- **cudaEvent:** repeatable per-kernel timing measurements.